SK SAHIL

Engineer Portfolio: Complete Architecture, Technical Guide & HR Pitch

A comprehensive walkthrough explaining project design, modern tech stack, tool mechanics, step-by-step tutorial, and recruiter presentation strategy.

Author: Sk Sahil | **Domain:** Computer Science & Cybersecurity
Repository: github.com/Sahil-9879 | **Live Site:** sk_sahil_portfolio.vercel.app

Executive Overview

This official documentation provides an in-depth, production-level breakdown of **Sk Sahil's Developer Portfolio**. Unlike standard drag-and-drop website builders or bloated theme templates (such as WordPress, Framer, or Elementor), this portfolio was architected from first principles using industry-standard engineering tools: **React 19, Vite, Tailwind CSS v4, Framer Motion, and Linux CLI concepts**.

It serves a dual purpose: first, as a **reusable technical tutorial** detailing how to construct a high-performance web application from scratch; and second, as a **recruiter-facing pitch guide** enabling candidate Sk Sahil to articulate technical decisions, system tradeoffs, and software architecture during HR and technical interviews.

1. Explaining the Project to HR & Recruiters

1.1 The 60-Second Interview Elevator Pitch

> *"When an HR manager asks: 'Can you tell me about a project you built recently?'"*

Suggested Answer:
"I designed and built my portfolio from scratch as a high-performance single-page web application using **React, Vite, and Tailwind CSS v4**. Instead of using generic templates, I engineered it with a **developer-focused UI** inspired by tools like VS Code and Linux terminals. Key engineering highlights include a custom-interpolated expanding horizontal navigation bar, an embedded interactive Linux modal terminal with command parsing, and a single-source-of-truth data structure. I also configured direct PDF resume downloads, responsive design, and automated continuous deployment to **Vercel** via GitHub CI/CD pipelines."

1.2 Why Build an 'Engineer-Grade' Application?

Most student portfolios use bright, colorful agency themes or heavy Framer templates that suffer from bloated JavaScript bundles, slow load times, and poor maintainability. Here is how to contrast your work:

Dimension	Generic Template (Framer/WP)	Sk Sahil's Portfolio (Custom Engineered)

Performance	Slow (3MB+ bundle size, heavy DOM scripts)	**Sub-second load** (~110KB gzip bundle, Vite optimized)
Architecture	Hardcoded HTML or visual editor nodes	**Modular React components** + strict separation of concerns
Interactivity	Basic hover effects or scroll triggers	**Interactive Linux CLI terminal** modal with full command history
Maintainability	Changes require visual builder tools	**Centralized data file** (`data.js`); edits take seconds
Design Tone	Flashy agency gradients & floating blobs	**Minimal slate dark mode** inspired by developer tools

2. Tools & Technologies Used (Detailed Breakdown)

When HR asks *'What tools did you use and why?'*, use this detailed reference:

Tool & Name	What Is That Tool?	Why Was It Chosen & How Is It Used Here?
React 19 (Core UI Library)	An open-source JavaScript library developed by Meta for building user interfaces based on components.	Used as the foundation of the app. Enables splitting the website into reusable components (`Navbar`, `ExpandingNav`, `Terminal`, `Contact`). State hooks (`useState`, `useCallback`) manage active tabs and terminal state.
Vite 8 (Build Tool)	A modern, ultra-fast frontend build tool that uses native ES modules to serve code instantly during development.	Replaced legacy Webpack. Provides Instant Hot Module Replacement (HMR) under 600ms and outputs highly compressed bundle chunks (`dist/`) for production.
Tailwind CSS v4 (Styling Engine)	A utility-first CSS framework that compiles utility classes directly into lightweight CSS stylesheet.	Configured with custom `@theme` variables in `src/index.css` to define the Midnight Slate color palette, custom fonts (Inter & JetBrains Mono), and glow effects without external design libraries.
Framer Motion (Animation)	A production-grade motion library for React that simplifies complex UI animations and layout transitions.	Powers smooth modal entrance/exit transitions for the Linux Terminal modal (`AnimatePresence`, `motion.div`) with hardware-accelerated 60fps performance.
Lucide React & Custom SVGs	Lightweight SVG icon collection tailored for clean UI interfaces.	Provides standard icons (Mail, FileText, MapPin, Terminal). Custom brand SVGs were engineered for GitHub, LinkedIn, LeetCode, and Vercel in `BrandIcons.jsx`.

Git & GitHub (Version Control)	Git tracks code revisions locally; GitHub hosts the remote repository on the cloud.	Stores code under `Sahil-9879/sk_sahil_portfolio`. Keeps clean commit history (`git commit`, `git push`) tracking every milestone.
Vercel Cloud (Hosting & CI/CD)	A global cloud platform optimized for static frontend frameworks with global CDN distribution.	Connected to GitHub repository. Automatically rebuilds and deploys the site every time code is pushed to the `main` branch.

3. System Architecture & Folder Layout

The codebase enforces strict **Separation of Concerns (SoC)**:

```
sk-sahil-portfolio/
■■■ public/
■   ■■■ resume.pdf                # Direct PDF download file
■■■ src/
■   ■■■ assets/                    # Static media assets
■   ■■■ components/
■   ■   ■■■ icons/
■   ■   ■   ■■■ BrandIcons.jsx      # SVG components (GitHub, LinkedIn, LeetCode, Vercel)
■   ■   ■■■ layout/
■   ■   ■   ■■■ Navbar.jsx          # Sticky navigation bar with logo & quick links
■   ■   ■■■ sections/
■   ■   ■   ■■■ Profile.jsx         # Hero section & bio summary
■   ■   ■   ■■■ About.jsx           # Detailed background & education timeline
■   ■   ■   ■■■ Projects.jsx        # Project cards grid with live & source links
■   ■   ■   ■■■ TechStack.jsx       # Categorized skills grid
■   ■   ■   ■■■ Contact.jsx         # Contact cards with hover glow effects
■   ■   ■■■ ExpandingNav.jsx        # Signature expanding flex navigation
■   ■   ■■■ ContentPanel.jsx        # Dynamic tab switcher container
■   ■   ■■■ Terminal.jsx            # Linux-style modal terminal window
■   ■■■ constants/
■   ■   ■■■ data.js                 # SINGLE SOURCE OF TRUTH (All content stored here)
■   ■■■ hooks/
■   ■   ■■■ useTerminal.js          # Custom React hook managing terminal CLI state
■   ■■■ App.jsx                     # Root application container wiring all components
■   ■■■ index.css                   # Tailwind CSS v4 @theme design tokens
■   ■■■ main.jsx                    # React 19 entry point mounting to DOM
■■■ index.html                      # HTML5 page head with Google Fonts & SEO meta tags
■■■ package.json                    # Dependencies & build scripts
■■■ vite.config.js                  # Vite configuration & Tailwind plugin wiring
```

4. Step-by-Step Build Tutorial

Step 1: Scaffold Project with Vite & Install Dependencies

Run the following terminal commands to create an empty Vite React project and install styling/animation packages:

```
# Create Vite project in current directory
npx -y create-vite@latest ./ --template react

# Install core dependencies
npm install tailwindcss @tailwindcss/vite framer-motion lucide-react
```

Step 2: Configure Tailwind v4 Theme Tokens (`src/index.css`)

Tailwind v4 uses CSS `@theme` blocks rather than JS configuration files. Define midnight colors and custom font families:

```
@import "tailwindcss";

@theme {
  --color-bg-primary: #0a0e17;
  --color-bg-card: #111827;
  --color-bg-card-hover: #1f2937;
  --color-text-primary: #f3f4f6;
  --color-text-secondary: #9ca3af;
  --color-accent: #3b82f6;
  --color-border: #1f2937;
  --font-sans: 'Inter', sans-serif;
  --font-mono: 'JetBrains Mono', monospace;
}
```

Step 3: Centralize Data Layer (`src/constants/data.js`)

Decouple content from UI components so edits never break JSX markup. Example `PERSONAL` and `CONTACT_LINKS` object structure:

```
export const PERSONAL = {
  name: 'Sk Sahil',
  role: 'Tech & Cybersecurity Enthusiast',
  college: 'B.Tech Computer Science',
  email: 'sksahil01018@gmail.com',
  github: 'https://github.com/Sahil-9879',
  linkedin: 'https://www.linkedin.com/in/sk-sahil-061a5a373/',
  leetcode: 'https://leetcode.com/u/sahil_0205/',
  resume: '/resume.pdf',
};
```

Step 4: Build Custom React Hooks (`src/hooks/useTerminal.js`)

Implement terminal command parsing (`help`, `about`, `projects`, `skills`, `contact`, `resume`) and direct file downloads:

```
if (trimmed === 'resume') {
  const a = document.createElement('a');
  a.href = PERSONAL.resume;
  a.download = 'Sk_Sahil_Resume.pdf';
  a.click();
}
```

Step 5: Wire Resume Download Links

Add the HTML5 `download="Sk_Sahil_Resume.pdf"` attribute to all resume anchor tags across `Navbar.jsx`, `Profile.jsx`, and `Contact.jsx` to ensure seamless, direct downloads when users click.

Step 6: Version Control & Deploy to Vercel

```
# Initialize Git & commit changes
git init
git add -A
git commit -m "initial commit: Sk Sahil Portfolio"

# Push to GitHub
git remote add origin https://github.com/Sahil-9879/sk_sahil_portfolio.git
git branch -M main
git push -u origin main
```

Connect the repository on **Vercel.com**. Vercel automatically runs `npm run build` and serves the app on global edge servers.

5. HR & Technical Interview Q&A; Cheatsheet

Q1: Why did you build this portfolio yourself instead of using template platforms like WordPress or Framer?

Answer: As a Computer Science student specializing in cybersecurity and software engineering, I wanted my personal site to reflect real code craft. Building it with React and Tailwind gave me complete control over performance (sub-second load time), accessibility, responsive layout behavior, and security best practices without third-party tracker bloat.

Q2: What is the most unique feature of your portfolio, and how did you implement it?

Answer: The interactive Linux-style modal terminal. I created a custom React hook (`useTerminal`) that maintains command history, handles keyboard events (like Up/Down arrow navigation and Enter key execution), and parses custom commands to dynamically switch sections or trigger direct resume PDF downloads.

Q3: How do you handle code updates when you want to change your bio, skills, or projects?

Answer: I engineered a single-source-of-truth data pattern (`data.js`). Content is completely separated from rendering code. Updating a project or skill requires editing a single Javascript object without risking JSX layout breakages.

Q4: How did you ensure fast loading speeds and smooth responsiveness?

Answer: I used Vite 8 for fast bundle splitting (~110KB gzipped JS), Tailwind CSS v4 for zero-runtime utility styling, and Framer Motion for hardware-accelerated GPU transitions. The layout uses CSS Grid and Flexbox for fluid responsiveness across mobile, tablet, and desktop viewports.

Q5: How is continuous deployment configured for your portfolio?

Answer: I connected my GitHub repository (`Sahil-9879/sk_sahil_portfolio`) to Vercel's CI/CD pipeline. Every git push to the `main` branch triggers an automated build verification and deploys the live production bundle in seconds.